

SMART Q - QUEUE MANAGEMENT SYSTEM WITH LIVE CROWD PREDICTION

PROJECT REPORT

Submitted to

ST JOSEPH'S UNIVERSITY

in partial fulfillment of the requirements for the degree of

Bachelor of Computer Application

by

Abhay Krishna (233BCAB05)

Nishan Rosary S (233BCAB11)

Stephen Fleming (233BCAB24)

under the guidance of

Ms Junaida Nallakkandy

Assistant Professor



School of Information Technology

Department of Computer Science

St Joseph's University

MARCH 2026



ST JOSEPH'S UNIVERSITY

A Public-Private-Partnership University under RUSA 2.0 of MHRD
(Government of India),

established by the Karnataka Govt. Act No. 24 of 2021



Laboratory Certificate

This is to certify that Smt. / Sri. _____
has satisfactorily completed the course of laboratory assignments in

_____ prescribed by St Joseph's University for the _____ Semester degree course
in BCA for the year 20__.

Signature of the Guide

Department Coordinator

Examiners:

Name: _____

Reg No.: _____

Date of Exam: _____

DECLARATION

I hereby declare that the work presented in this project report entitled “Smart’Q: Queue Management System with Live Crowd Prediction” is based on the original project work carried out by me and my team under the supervision of Ms Junaida Nallakkandy, Assistant Professor, Department of Computer Science, St Joseph’s University. I further declare that the work presented in this report, either in full or in part, has not been submitted previously for the award of any other degree, diploma, or certification.

Place : **Bangalore**

Abhay Krishna

Date :

Nishan Rosary

Stephen Fleming

Contents

ACKNOWLEDGEMENT	i
ABOUT THE UNIVERSITY	ii
SCHOOL OF INFORMATION TECHNOLOGY	iv
DEPARTMENT OF COMPUTER SCIENCE	v
LIST OF FIGURES	viii
1 INTRODUCTION	1
1.1 Problem Definition	3
1.2 Objective	3
2 DATA FLOW DIAGRAM - DFD	4
2.1 Introduction	4
2.2 External Entities	5
2.2.1 Customer	5
2.2.2 Administrator	5
2.2.3 Machine Learning Service	6
2.2.4 Email Service (SMTP Server)	6
2.3 Level 0 DFD – Smart’Q System	6
2.4 Data Flows	7
2.5 Level 1 DFD Breakdown	7
2.5.1 User Interface (Frontend)	7
2.5.2 Backend API (Node.js / Express)	8
2.5.3 Database Management (MongoDB)	8
2.5.4 Machine Learning Integration	8
2.5.5 Email Automation Module	8
2.6 Data Stores	8
3 Entity Relationship (ER) Diagram	10
3.1 Introduction	10
3.2 Entities	11
3.2.1 User Entity	11
3.2.2 Event Entity	12
3.2.3 Queue Entity	12

3.2.4	EmailNotification Entity	13
3.3	Relationships	13
3.3.1	User Joins Queue (One-to-Many)	14
3.3.2	Event Has Queue (One-to-Many)	14
3.3.3	User Receives EmailNotification (One-to-Many)	14
3.3.4	Queue Triggers EmailNotification (One-to-Many)	14
4	Codes and Screenshot	15
4.1	Frontend Implementation	15
4.1.1	Explanation	15
4.1.2	Files	15
4.2	Backend Implementation	29
4.2.1	Explanation	29
4.3	Machine Learning Implementation	42
4.3.1	Explanation	42
4.4	Email SMTP Implementation	48
4.4.1	Explanation	48
5	Conclusion	52
5.1	System Deployment	53
5.2	Source Code Repository	54
	BIBLIOGRAPHY	55
	APPENDICES	56
	Appendix A: Website Screenshots	56
	Appendix B: Code Screenshots	58

ACKNOWLEDGEMENT

The successful execution of this project would not have been achievable without the unwavering support and guidance of numerous individuals. We express our profound gratitude to the Almighty for his blessings, guidance, and strength that enabled us to successfully complete this project. We are sincerely thankful to Rev. Dr. Victor Lobo S.J., Vice Chancellor of St. Joseph's University, and Dr. A. M. Bojamma, Dean of the School of Information Technology, for providing us with the opportunity and the academic environment to carry out this work. We take this opportunity to express our deepest sense of gratitude to our guide, Ms. Junaida Nallakkandy, Assistant Professor, Department of Computer Science, for her invaluable guidance, constant encouragement, and constructive suggestions throughout the development of this project. Her support has been instrumental in shaping our work.

We also extend our thanks to all the faculty members of the Department of Computer Science for their support and cooperation.

Finally, we express our heartfelt gratitude to our parents and friends for their continuous encouragement and moral support.

ABOUT THE UNIVERSITY

St Joseph's University (SJU) is a Jesuit university at the heart of Bengaluru, the Silicon city of India. Established in 1882 by Paris foreign Fathers, the management of the college was handed over to the Jesuit order (Society of Jesus) in 1937. The college was first affiliated to the university of Madras and later to the Mysore and Bangalore universities. In 1986, St Joseph's College became the first affiliated college in Karnataka to offer postgraduate courses. In 1988, it became the first college in Karnataka to get a research center and in 2005, it was one of five colleges in Karnataka that was awarded academic autonomy. In February 2021, St Joseph's University bill was presented in the Karnataka Legislative Assembly and was subsequently passed by the Legislative Assembly and Karnataka Legislative Council. The college received its University status on 2nd July 2022 and was inaugurated as India's first public-private university by the President of India, Smt. Droupadi Murmu on 27 September 2022.

Milestones

St Joseph's University (SJU) is a Jesuit University in the heart of Bengaluru, the Silicon City of India, established in 1882 by Paris Foreign Fathers. The college management was handed over to the Jesuits (Society of Jesus) in 1937. The college was first affiliated to the University of Madras, and later with Mysore and Bangalore Universities.

1986 – Became the first affiliated college in Karnataka to offer postgraduate courses

1998 – Became the first college in Karnataka to get a research center

2005 – College was granted autonomous status

February 2021 – Bill presented in Karnataka Legislative Council

02 July 2022 – Received University status

27 September 2022 – Inaugurated as India's first Public-Private Partnership University by the Hon'ble President of India, Smt. Draupadi Murmu

Vision To form women and men for and with others, who through

holistic education, strive for a just, secular, democratic, and ecologically sensitive society which empowers the poor, the oppressed and the marginalized Mission In keeping with the Jesuit heritage, the university aims at an integral formation of the staff and the students, to be men and women who will be agents of societal change, by enabling them to attain academic and human excellence in a teaching-learning environment that fosters intellectual curiosity, ceaseless enquiry, personal integrity, social commitment, creativity, critical thinking, and innovation.

SCHOOL OF INFORMATION TECHNOLOGY

The School of IT consists of the Department of Computer Science and the Department of Advanced Computing. The School of Information Technology (SoIT) offers programs related to: Data analytics, Computer science, Computer applications, Domain-specific applications. SoIT focuses on holistic learning that helps students make major contributions to the IT industry and serve society at large. The school has more than 2000 students and 40 committed faculty members, along with many visiting professors and working professionals from industry, research, and development organizations. Students are provided opportunities to apply acquired knowledge to solve real-world problems and gain research experience. The placement record of SIT is always exceptional

DEPARTMENT OF COMPUTER SCIENCE

The Department was founded in 1986, offering BSc in Computer Science along with Physics and Mathematics. Continuous programmes were developed and administered, serving the needs of industrial computer scientists, researchers, and computer application specialists for whom the discipline of computing sciences provides indispensable tools with newer programmes being added. The Department conducts a Board of Studies every year and offers the syllabus according to the Industry requirements and demands. BCA is an undergraduate programme for candidates wishing to delve into the world of Computer Languages. Being one of the most popular options to get started with a career in Information Technology, this programme gives you an insight into the world of computers and their applications. MSc Computer Science is a postgraduate programme provisioning to a group of candidates drawn to probe deeper into the realm of computer science. It aims to prepare young professionals for challenging business opportunities in the IT sector.

The Department also assists research scholars at various universities such as Bharathiar University, Sri Krishna Devaraya University, and Mother Teresa University. All faculty members have completed their MPhil and are on the verge of completing their PhD. Both members of the staff and students are actively involved in the research. The students are required to prepare a term paper that is a culmination of a certain amount of research performed by the student on a topic of their choice as approved by the Department. The student is guided continuously during the process by the teachers. All final-year students will have to develop and complete a project for an organization outside the College. The students need to have a deep awareness of

the existing environment and produce workable solutions, ensuring that they have sufficient exposure and preparedness for a challenging career in the field. The Department organizes conferences, workshops, Intercollegiate IT fests, FDPs, Guest lectures, Expert talks, and Panel discussions on new trends in computer science and Information technology.

Vision

The Vision of the Department is to ensure that our students have a deep and analytical understanding of the field and to enable them to use their immense potential to enhance the quality of human life by contributing to the field of Technology, preparing them for their Career in the Industry, and to the needs of the Society with Ethical responsibilities.

Mission

The Department offers quality education in the Science of Computing by providing courses with Computer Applications, Courses in combination with Physical Science and Courses with Specialization and by focusing on the Personal, Emotional and Character development.

List of Figures

2.1	Data Flow Diagram	5
3.1	Entity Relationship Diagram	11
4.1	Smart-Q Landing Interfaces	16
4.2	Smart-Q Customer Authentication	17
4.3	Smart-Q Admin Authentication	17
4.4	Administrator Dashboard Overview	22
4.5	Customer Dashboard Showing Queue Position	23
4.6	Smart-Q Settings Interface	24
4.7	Smart-Q Dark Mode Theme	25
4.8	Mongo DB Atlas	34
4.9	Queue Management Interface for Administrators	37
4.10	Service Counter Management Interface	37
4.11	Event Scheduler Interface	39
4.12	Archived Event History Management	39
4.13	Machine Learning Predictions on Customer Dashboard	42
4.14	Machine Learning Predictions Dashboard	44
4.15	Machine Learning Predictions on Customer Dashboard	45
4.16	OTP Verification Email	49
4.17	Queue Registration Confirmation Email	50
5.1	Smart-Q Guest Mode Dashboard Page	56
5.2	Smart-Q Waiting Queue Management Page	57
5.3	Admin Notification Interface	57
5.4	CI/CD Pipeline Configuration for Smart-Q Deployment	58
5.5	Backend Routes for Machine Learning Prediction	58

CHAPTER 1

INTRODUCTION

Queue management is an essential part of service-based environments such as hospitals, banks, educational institutions, and government offices. In many of these places, people spend a significant amount of time waiting in long queues to access services. The traditional queue system often leads to overcrowding, inefficient service flow, and frustration among customers. Long waiting times also reduce productivity and create unnecessary workload for staff members.

The waiting time experienced by customers in queues usually depends on several factors such as:

- Number of customers waiting for service
- Availability of service counters or staff
- Service time required for each customer
- Event or appointment scheduling
- Peak service hours and crowd density
- Efficiency of the queue handling process

There are many additional factors that influence queue delays and service efficiency. The key observation is that most of these problems arise due to the lack of proper queue monitoring, prediction, and management systems. Traditional manual queue systems do not provide real-time updates or estimated waiting times, which makes it difficult for users to plan their time effectively. With the advancement of web technologies, data analysis, and machine learning, it is possible to build intelligent

systems that can automate and optimize the queue management process. A smart queue management system can help users join queues remotely, monitor their queue position in real time, and receive accurate waiting time predictions. Such systems not only improve customer satisfaction but also enhance operational efficiency for service providers.

This project proposes Smart Q, a web-based Queue and Event Management System designed to modernize the traditional queue handling process. Smart Q allows users to join queues remotely through a web application instead of physically waiting in line. Once registered, users receive real-time updates about their queue position and estimated waiting time. This enables them to manage their time efficiently and arrive only when their turn is approaching. The system consists of two primary modules: the Admin Module and the Customer Module. The Admin module enables administrators to create and manage events, monitor queue flow, manage service counters, and analyze system performance. Administrators can track the number of users currently in the queue, view real-time statistics, and efficiently control the lifecycle of events. The Customer module allows users to register for events, join queues, receive confirmation notifications, and track their queue status in real time through the web interface.

A key feature of Smart Q is the integration of a Machine Learning model based on the Random Forest algorithm to predict estimated waiting time. The system analyzes historical queue data such as the number of users, service duration, and processing patterns to generate accurate waiting time predictions. This predictive capability improves user experience and assists administrators in making better operational decisions. The model can also retrain automatically when new data becomes available, allowing continuous improvement in prediction accuracy.

The Smart Q system is developed using modern full-stack technologies. The frontend is built using HTML, CSS, JavaScript, and React, providing a responsive and user-friendly interface. The backend is implemented using Node.js and Express.js to manage server-side logic and APIs. The system uses MongoDB or MySQL for storing queue and user data. Additionally, a Python-based machine learning microservice using Scikit-learn performs predictive analysis. Real-time queue updates are enabled through Socket.IO, allowing instant communication between the server and users.

1.1 Problem Definition

In many public service environments, customers are required to wait in long physical queues before accessing services. These traditional queue systems often lead to overcrowding, inefficient service management, and increased waiting times. Users are unable to track their queue position or estimate when their turn will arrive, resulting in wasted time and frustration.

Service administrators also face difficulties in monitoring queue flow, predicting waiting times, and managing events efficiently. Therefore, there is a need for an automated system that can manage queues digitally, provide real-time updates, and predict waiting times accurately.

An intelligent queue management system is required to reduce physical waiting time, improve service efficiency, and enhance the overall experience for both users and administrators.

1.2 Objective

The main objective of this project is to design and develop a Smart Queue Management System that improves traditional queue handling processes using modern web technologies and machine learning.

The key objectives of the system are:

- To allow users to join queues remotely through a web application
- To provide real-time updates about queue position and waiting time
- To enable administrators to manage events, counters, and queue flow efficiently
- To analyze historical queue data for predicting waiting time
- To integrate a machine learning model using the Random Forest algorithm for accurate predictions
- To improve overall service efficiency and reduce physical waiting time

CHAPTER 2

DATA FLOW DIAGRAM - DFD

2.1 Introduction

A Data Flow Diagram (DFD) is a graphical representation used to illustrate how data moves within a system. It describes the flow of information between external entities, internal processes, and data stores. The DFD helps in understanding how input data is transformed into output through different system components.

In the Smart'Q Queue and Event Management System, the DFD explains how customers interact with the system to join queues, how administrators manage events and service counters, and how the backend processes requests. It also demonstrates the integration of additional services such as the Machine Learning prediction module and Email notification service. By analyzing the data flow, it becomes easier to understand the internal functioning of the system and how different modules communicate with each other.

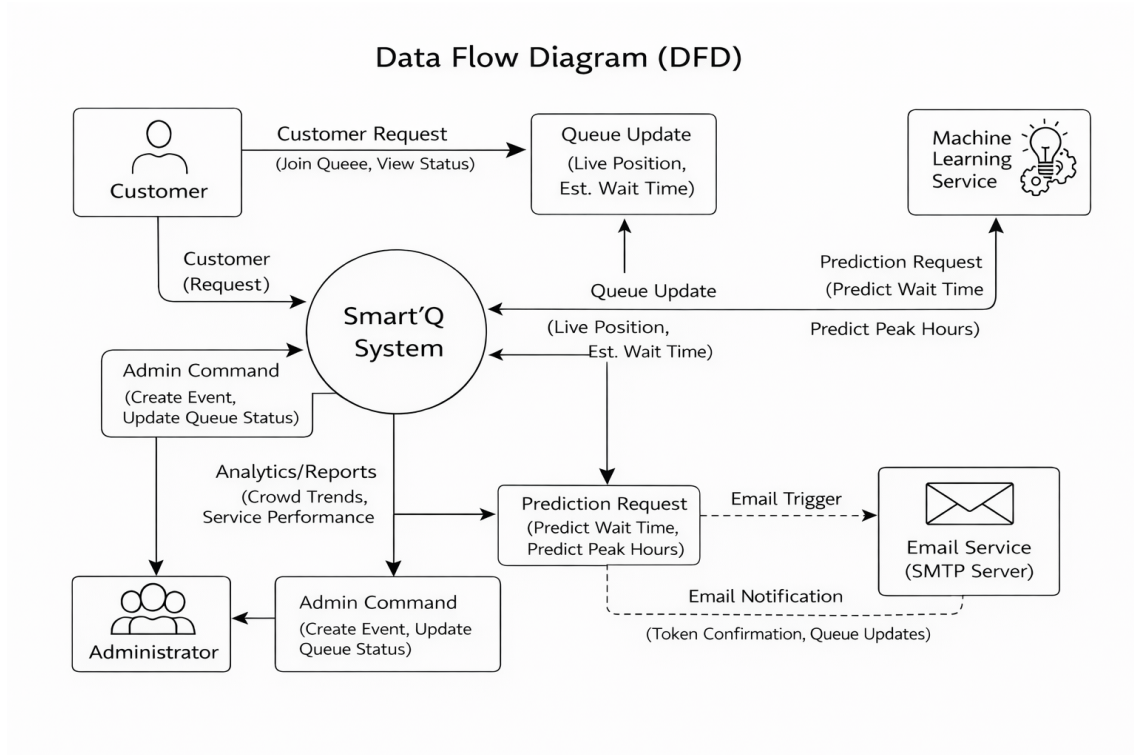


Figure 2.1: Data Flow Diagram

2.2 External Entities

External entities represent the sources and destinations of data that interact with the Smart'Q system.

2.2.1 Customer

The Customer represents the end users of the system who join queues and monitor their queue status. Customers can register for events, receive token numbers, and track their queue position in real time. They also receive automated email notifications when their queue status changes or when their turn is approaching.

2.2.2 Administrator

The Administrator represents authorized personnel responsible for managing the system operations. Administrators can create events, manage service counters, monitor queue flow, and update token statuses such as waiting, serving, or completed. They

also monitor system analytics and overall queue performance.

2.2.3 Machine Learning Service

The Machine Learning Service is a separate Python-based microservice integrated with the backend. It processes historical queue data and predicts important parameters such as estimated waiting time, queue length, and expected crowd levels. These predictions help both administrators and users to better manage their time and resources.

2.2.4 Email Service (SMTP Server)

The Email Service is an external SMTP-based service used for sending automated email notifications to customers. Whenever a token is confirmed, updated, or nearing service time, the backend triggers this service to send confirmation emails to users.

2.3 Level 0 DFD – Smart’Q System

At Level 0, the entire Smart’Q application is represented as a single process called the Smart’Q System. This level provides a high-level overview of how the system interacts with external entities.

The Smart’Q system performs the following major functions:

- Processing customer requests such as joining queues and checking queue status
- Handling administrator commands for managing events and queues
- Providing real-time queue updates using WebSocket communication
- Communicating with the Machine Learning service for waiting time prediction
- Triggering automated email notifications for token updates
- Managing and storing system data in the database

This level mainly focuses on how data flows between the system and its external entities without detailing internal processes.

2.4 Data Flows

Data flows represent the movement of information between entities, processes, and data stores within the Smart'Q system.

The major data flows in the system include:

- Customer Request – Generated when a customer joins a queue or checks queue status.
- Queue Update – Real-time updates sent from the backend to customers using WebSocket communication.
- Admin Command – Instructions sent by the administrator to update queue status or manage events.
- Prediction Request – Queue data sent from the backend to the Machine Learning service for waiting time estimation.
- Prediction Result – Predicted waiting time returned from the Machine Learning service.
- Email Trigger – Request generated by the backend to initiate email notifications when token status changes.
- Email Notification – Automated email sent to customers confirming token status updates.

2.5 Level 1 DFD Breakdown

At Level 1, the Smart'Q system is divided into several internal processes to provide a more detailed view of system operations.

2.5.1 User Interface (Frontend)

The User Interface is developed using React and handles interactions between users and the system. Customers can join queues, track their position, and view waiting time estimates. Administrators can manage events, monitor queue status, and control service counters through the interface.

2.5.2 Backend API (Node.js / Express)

The Backend API processes system logic and handles communication between the frontend, database, and external services. It manages authentication, queue operations, event management, machine learning integration, and email automation.

2.5.3 Database Management (MongoDB)

The Database stores all system-related information including user details, event records, queue entries, timestamps, and prediction-related data. This data is used for queue management and machine learning analysis.

2.5.4 Machine Learning Integration

The Machine Learning module communicates with a Python-based microservice that analyzes historical queue data. It predicts waiting time and other queue metrics using the Random Forest algorithm.

2.5.5 Email Automation Module

The Email Automation module is responsible for sending automated notifications to customers. Using the SMTP protocol, it sends confirmation emails when tokens are generated, updated, or nearing service time.

2.6 Data Stores

The Smart'Q system uses multiple data stores to manage and organize system data efficiently.

- Customer Data Store – Stores user information, login credentials, and customer profiles.
- Event Data Store – Stores details about events created by administrators.
- Queue Data Store – Stores token numbers, queue status, timestamps, and waiting time data.

-
- ML Model Store – Stores trained machine learning models used for prediction.
 - Email Log (Optional) – Stores records of sent emails for monitoring and auditing purposes.

CHAPTER 3

Entity Relationship (ER) Diagram

3.1 Introduction

An Entity Relationship (ER) Diagram is a conceptual representation of the database structure that illustrates the entities in a system and the relationships between them. It helps in designing and organizing the database by defining how data is stored, accessed, and connected across different components of the system.

In the Smart'Q Queue and Event Management System, the ER diagram represents the main entities involved in queue management such as Users, Events, Queue entries, and Email Notifications. These entities interact with each other to support system functionalities such as event creation, queue joining, queue monitoring, and automated notification services.

The ER diagram provides a clear understanding of how user information, event details, queue data, and notification records are stored and related within the system database.

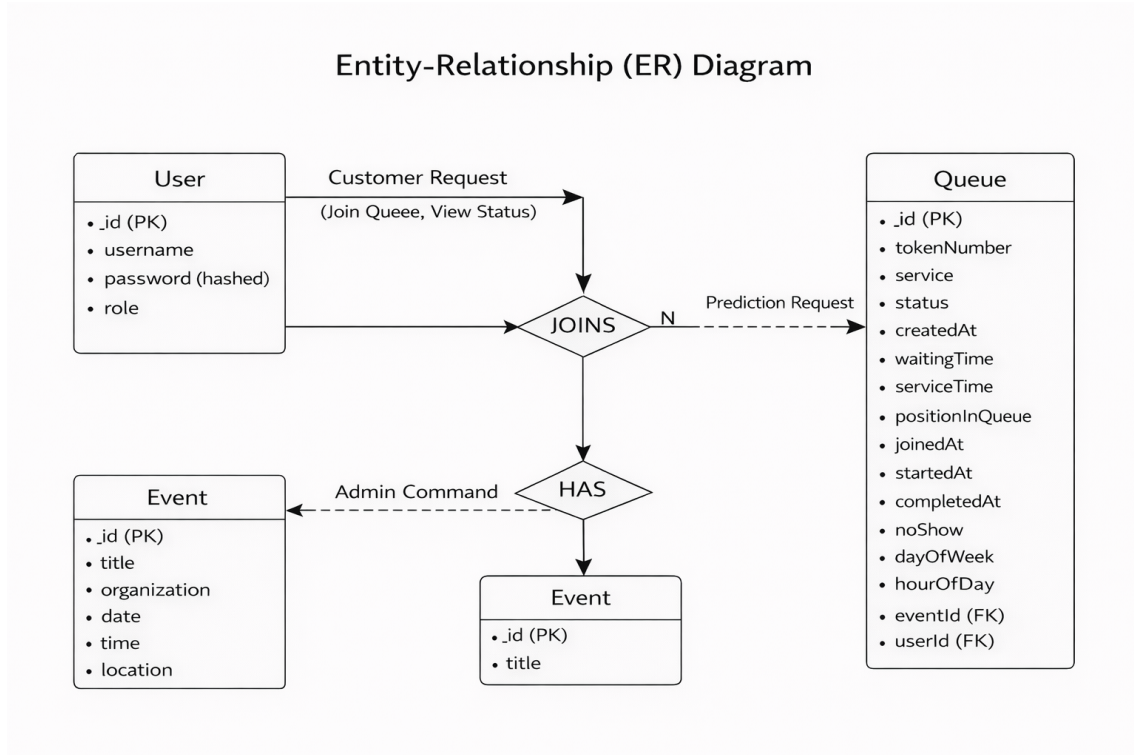


Figure 3.1: Entity Relationship Diagram

3.2 Entities

The Smart'Q system consists of several core entities that represent the primary data objects stored in the database.

3.2.1 User Entity

The User entity represents individuals who interact with the Smart'Q system. Users can either be customers who join queues or administrators who manage events and queue operations.

The User entity stores authentication details and user roles which determine the permissions within the system.

Attributes of the User entity include:

- `_id (Primary Key)` – Unique identifier for each user
- `username` – Name of the user

-
- email – Email address used for login and notifications
 - password – Encrypted password used for authentication
 - role – Defines whether the user is a customer or administrator

3.2.2 Event Entity

The Event entity represents the services or activities where queue management is required. Administrators can create events such as hospital appointments, bank services, college registrations, or government service sessions.

Each event contains information about the service schedule and location.

Attributes of the Event entity include:

- _id (Primary Key) – Unique identifier for each event
- title – Name of the event or service
- organization – Organization conducting the event
- date – Scheduled date of the event
- time – Scheduled time for the event
- location – Venue or service location
- createdAt – Timestamp when the event was created
- updatedAt – Timestamp when the event details were last updated

3.2.3 Queue Entity

The Queue entity represents the queue entries created when users join a particular event. Each queue entry corresponds to a token number assigned to a user.

The queue entity also tracks the status and timestamps of service progress.

Attributes of the Queue entity include:

- _id (Primary Key) – Unique identifier for each queue entry
- tokenNumber – Token number assigned to the user
- service – Type of service requested

-
- status – Current queue status (waiting, serving, completed, or no-show)
 - waitingTime – Estimated waiting time predicted by the system
 - serviceTime – Duration taken to serve the user
 - positionInQueue – Current position of the user in the queue
 - joinedAt – Timestamp when the user joined the queue
 - startedAt – Timestamp when service begins
 - completedAt – Timestamp when service is completed
 - eventId (Foreign Key) – References the associated event
 - userId (Foreign Key) – References the user who joined the queue

3.2.4 EmailNotification Entity

The EmailNotification entity is responsible for storing records of automated email notifications sent to users. These notifications inform users about token confirmations, queue status updates, or when their turn is approaching.

Attributes of the EmailNotification entity include:

- id (Primary Key) – Unique identifier for each notification record
- userId (Foreign Key) – References the user receiving the notification
- queueId (Foreign Key) – References the related queue entry
- emailStatus – Status of the email (sent, pending, failed)
- sentAt – Timestamp when the email notification was sent

3.3 Relationships

Relationships define how entities interact with each other within the database structure.

3.3.1 User Joins Queue (One-to-Many)

A User can join multiple queue entries across different events. However, each queue entry belongs to only one user.

Therefore, the relationship between User and Queue is One-to-Many (1:N).

3.3.2 Event Has Queue (One-to-Many)

An Event may contain multiple queue entries because many users can join the same event. However, each queue entry is associated with only one event.

Thus, the relationship between Event and Queue is One-to-Many (1:N).

3.3.3 User Receives EmailNotification (One-to-Many)

A User may receive multiple email notifications during their interaction with the system. For example, users may receive notifications when a token is generated, when the queue status changes, or when their turn is approaching.

Therefore, the relationship between User and EmailNotification is One-to-Many (1:N).

3.3.4 Queue Triggers EmailNotification (One-to-Many)

A Queue entry may trigger one or more email notifications depending on status updates and system events. For instance, notifications may be sent when the token is confirmed or when the queue progresses.

Thus, the relationship between Queue and EmailNotification can be One-to-Many (1:N).

CHAPTER 4

Codes and Screenshot

4.1 Frontend Implementation

4.1.1 Explanation

The Smart-Q frontend is developed using React and organized into modular layers including the application shell, page components for admin and customer views, shared UI components, API and socket services, helper utilities, styles, and testing scripts. The frontend communicates with the backend through REST APIs and Socket.IO for real-time queue updates. The architecture follows a state-driven navigation approach to manage page transitions and maintain session state.

4.1.2 Files

`src/index.js`

Code

```
const root = ReactDOM.createRoot(  
  document.getElementById( 'root ' )  
);  
  
root.render(  

```

```
<React.StrictMode>
<App />
</React.StrictMode>
);
```

Explanation

This file serves as the main entry point of the React application. It initializes the React DOM root and renders the main App component inside the HTML container element. React StrictMode is used to highlight potential problems in development. This file essentially bootstraps the entire frontend application.

src/App.jsx

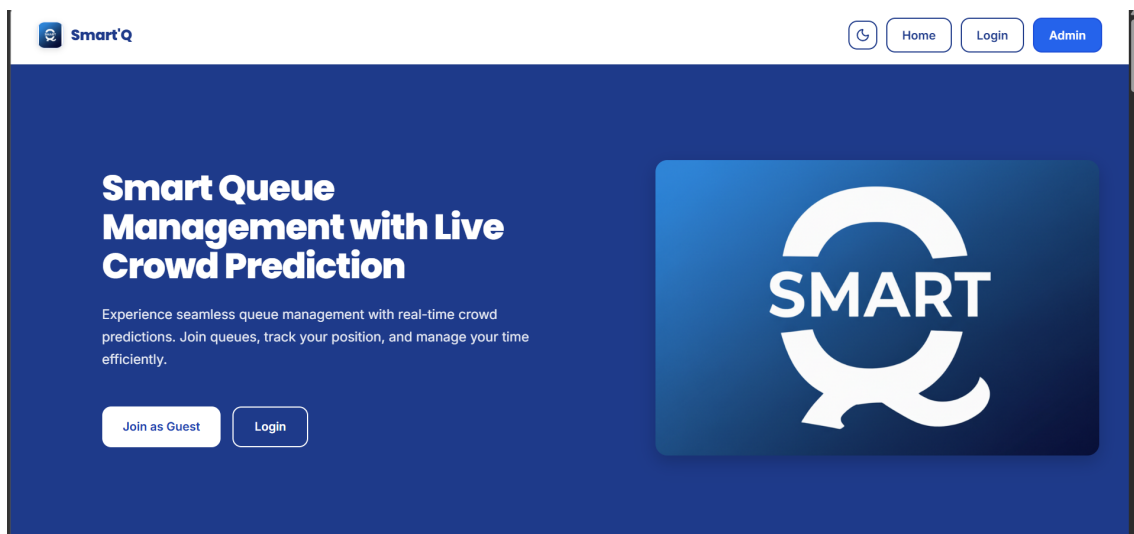


Figure 4.1: Smart-Q Landing Interfaces

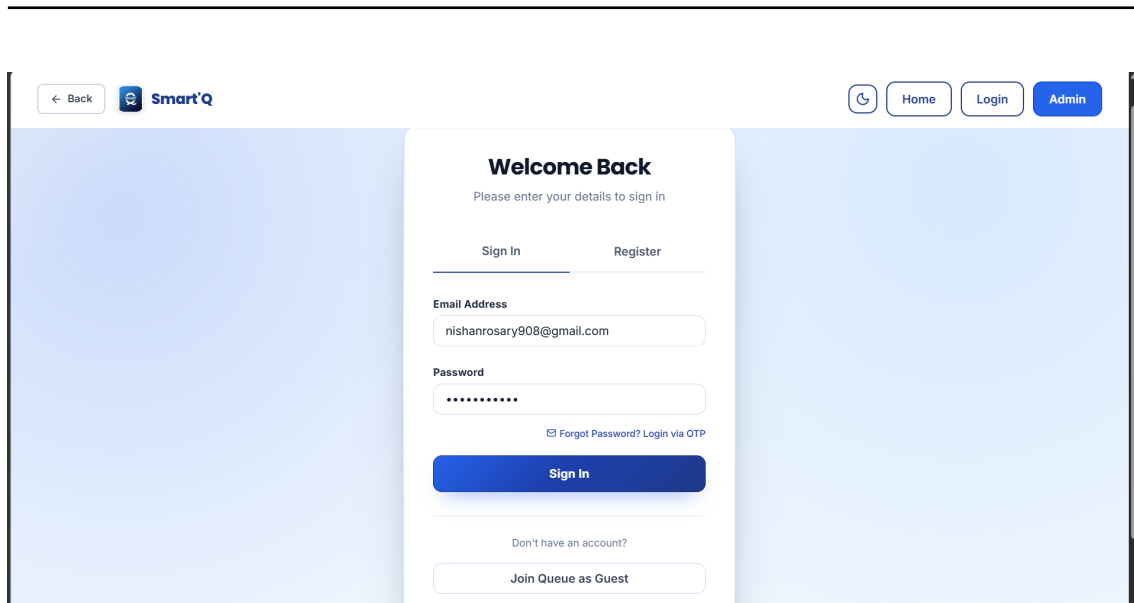


Figure 4.2: Smart-Q Customer Authentication

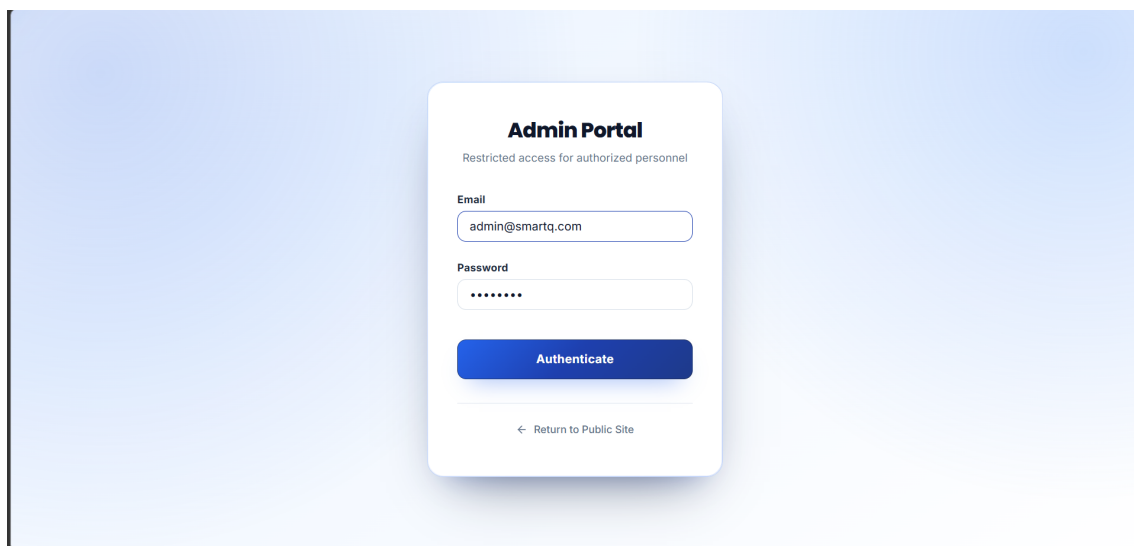


Figure 4.3: Smart-Q Admin Authentication

Code

```
const [currentPage, setCurrentPage] =
  useState("landing");

const navigate = (page, data = null) => {
  setCurrentPage(page);
}
```

```
};

const renderPage = () => {
  switch (currentPage) {
    case "landing":
      return <LandingPage />;

    case "admin-dashboard":
      return <AdminDashboard />;

    case "join-queue":
      return <JoinQueue />;
  }
};
```

Explanation

This component acts as the main application shell and navigation controller. It maintains the current page state and dynamically renders different components depending on the navigation state. It also handles session restoration, backend connectivity checks, and deep-link navigation for QR code access.

—

src/config/api.js

Code

```
const PROD_API_BASE_URL =
  "https://smartq-backend-b4jn.onrender.com";

const getApiBaseUrl = () => {
  ...runtime/build/local resolution ...
```

```
};
```

```
export const API_BASE_URL = getApiBaseUrl();
```

Explanation

This file determines the base URL used for all backend API calls. It dynamically resolves the correct endpoint based on the runtime environment such as local development or production deployment. This flexible configuration allows the frontend to seamlessly connect to different backend environments without code modification.

src/services/api.js

Code

```
const API = axios.create({
  baseURL: `${API_BASE_URL}/api`,
  withCredentials: true
});

API.interceptors.response.use(
  (r) => r,
  async (error) => {

    if (error.response?.status === 401) {

      const res =
        await API.post("/auth/admin/refresh");

      setAuthToken(res.data.accessToken);
    }
  }
);
```

```
return API(originalRequest);

}

}

);
```

Explanation

This module creates a centralized Axios API client for communicating with backend services. It attaches authentication tokens automatically and handles token refresh when a request receives a 401 unauthorized response. By centralizing API calls, the application ensures consistent error handling and authentication management across all components.

src/services/queueService.js

Code

```
socket = io(SOCKET_URL, {
  transports: [ "websocket", "polling" ],
  withCredentials: true
});

export const onQueueUpdate = (cb) => {
  getSocket().on("queue:update", cb);
};

export const joinQueue = async (...) =>
  axios.post(`${API_BASE_URL}/api/queue/join`);
```

Explanation

This service manages queue-related operations and real-time updates. It establishes a socket connection with the backend server to receive queue update events. The module also provides helper functions for joining queues and retrieving queue status through REST API calls.

src/socket.js

Code

```
const socket = io(API_BASE_URL, {
  transports: ["websocket"]
});

export default socket;
```

Explanation

This file creates a reusable Socket.IO client instance for real-time communication. It connects the frontend to the backend socket server and enables components to listen for live events such as queue updates. By exporting a shared socket instance, multiple components can access the same connection efficiently.

`components/admin/AdminDashboard.jsx`

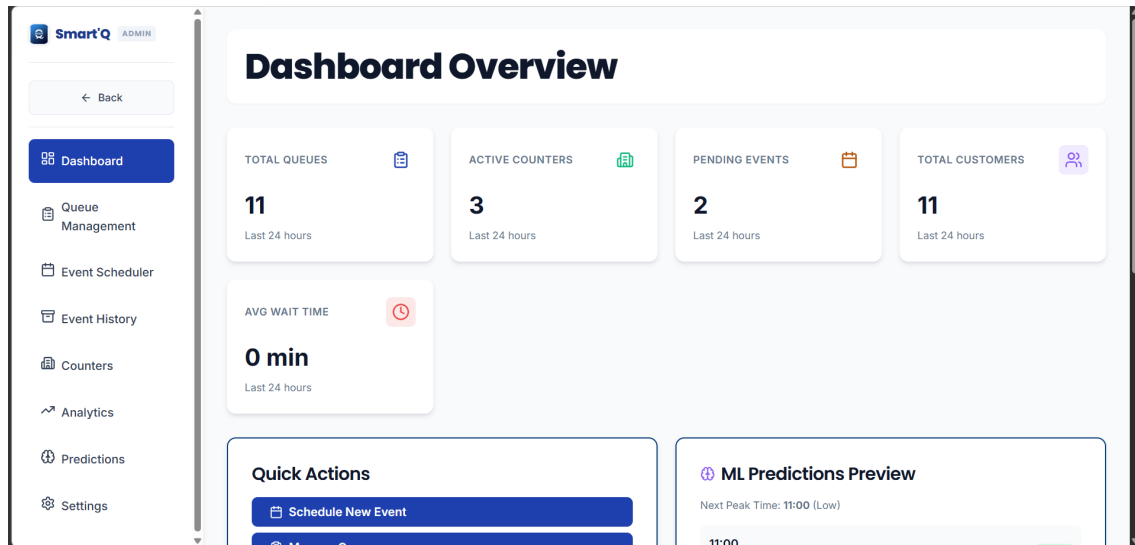


Figure 4.4: Administrator Dashboard Overview

Code

```
const [queueRes, eventsRes] =
await Promise.all([
  axios.get("/api/queue"),
  axios.get("/api/events")
]);

setStats({
  totalQueues: ...,
  pendingEvents: ...,
  averageWaitTime: ...
});
```

Explanation

This component represents the administrative dashboard used to monitor queue operations. It retrieves queue and event data from the backend and calculates summary statistics such as total queues and average waiting time. The dashboard

also integrates machine learning predictions for operational insights.

`components/customer/CustomerDashboard.jsx`

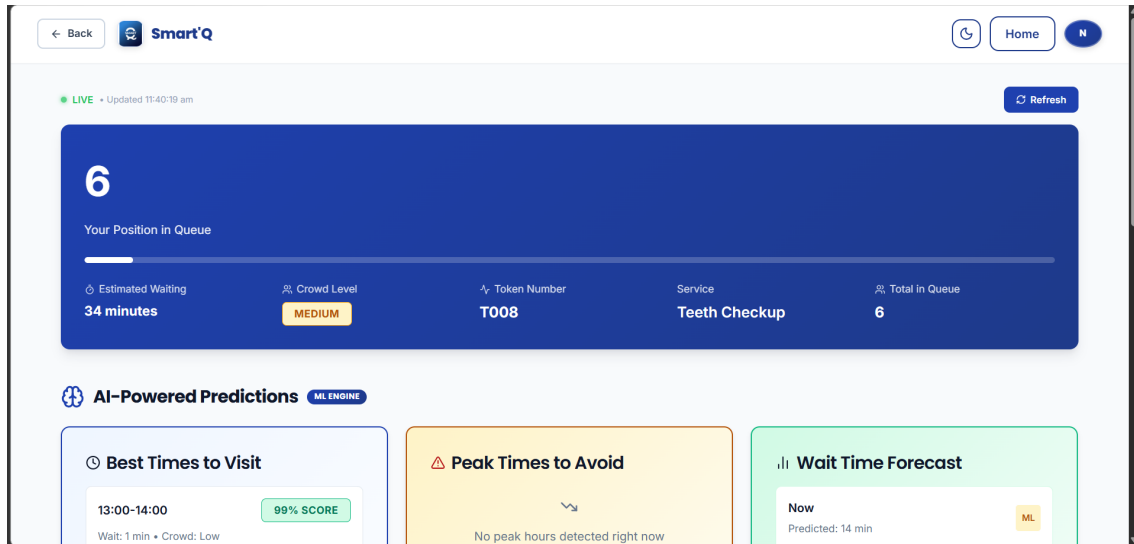


Figure 4.5: Customer Dashboard Showing Queue Position

Code

```
useEffect(() => {
  connectSocket();
  onQueueUpdate(handleQueueUpdate);
  fetchQueueStatus();
}, [activeToken]);

const data =
  await getQueueStatus(
    normalizedToken,
    activeService
  );
```

Explanation

This component provides the main interface for customers to track their queue status. It listens for real-time queue updates through Socket.IO and periodically retrieves the current queue state. The dashboard displays token status, waiting time estimates, and event details.

`components/shared/Header.jsx`

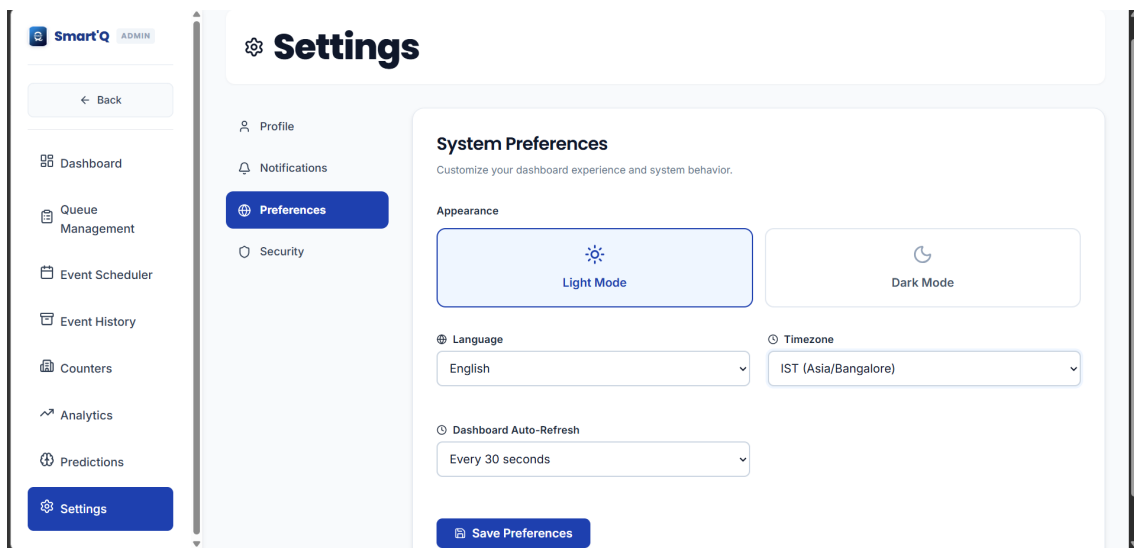


Figure 4.6: Smart-Q Settings Interface

Code

```
const toggleTheme = () => {

  localStorage.setItem(
    'smartq-theme',
    newTheme
  );

};

const handleLogout = () => {
```



```
onLogout?.();  
onNavigate?.( 'login ' );  
  
};
```

Explanation

This shared component provides the top navigation bar for both admin and customer interfaces. It includes controls for navigation, theme switching, and user logout. Theme preferences are stored in local storage so the interface retains the selected theme across sessions.

styles/global.css

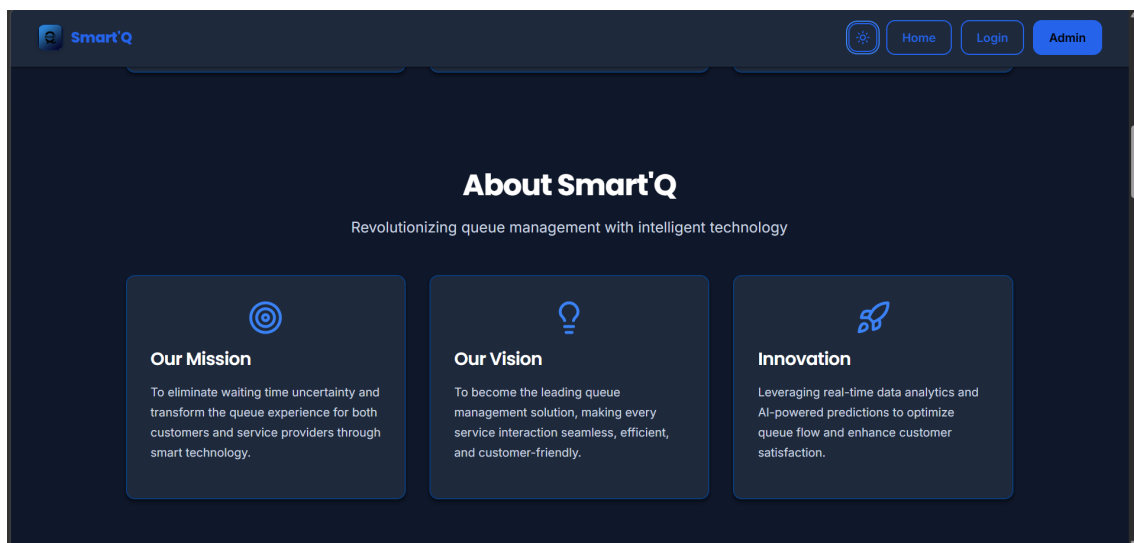


Figure 4.7: Smart-Q Dark Mode Theme

Code

```
:root {  
  ...design tokens...
```

```
}

[data-theme='dark']{
  ...dark theme tokens...
}

.btn-primary ,
.card ,
.badge ,
input {
  ... styles ...
}
```

Explanation

This stylesheet defines the global design system used across the application. It includes CSS variables for colors, typography, and layout spacing. Theme variables enable both light and dark interface modes, ensuring consistent styling throughout all frontend components.

webpack.config.js

Code

```
entry: './src/index.js',

plugins: [
  DefinePlugin ,
  HtmlWebpackPlugin ,
  CopyWebpackPlugin
],
```

```
devServer:{
  port:3000,
  proxy:[
    {
      context:[ '/api', '/socket.io' ],
      target:'http://localhost:5000 '
    }
  ]
}
```

Explanation

This configuration file controls the Webpack build process for the React application. It defines the entry point, plugins for HTML generation and environment variables, and development server settings. The proxy configuration allows frontend requests to be forwarded to the backend server during development.

package.json

Code

```
"scripts": {
  "start":
    "webpack serve --mode development",

  "build":
    "webpack --mode production",

  "test":
    "node ./tests/run-sanity.mjs",
```

```
"test:ui":  
"npm-run build &&  
node ./tests/run-browser-ui.mjs"  
}
```

Explanation

This file defines the project dependencies and runtime scripts used by the frontend application. It includes commands for starting the development server, building production bundles, and running automated tests. The configuration ensures that the development and deployment workflows remain consistent.

4.2 Backend Implementation

4.2.1 Explanation

The backend of the Smart-Q system is implemented using a modular Node.js and Express architecture integrated with MongoDB. The backend layer is responsible for managing authentication, queue lifecycle operations, event scheduling, role-based access control, rate limiting, and real-time updates through Socket.IO.

The backend also performs automated maintenance tasks such as archiving expired events and cleaning outdated queue records. The architecture follows a layered middleware structure and organized routing mechanism to ensure scalability, maintainability, and system security.

API Endpoints

The Smart-Q backend exposes a comprehensive set of RESTful API endpoints that enable communication between the frontend application, backend server, and machine learning microservice. These APIs handle authentication, queue management, event operations, prediction services, and system utilities.

1. Authentication APIs

These APIs manage user registration, login, and authentication-related operations.

- POST /api/auth/register – Register a new customer user
- POST /api/auth/login – Customer login
- POST /api/auth/admin/login – Administrator login
- POST /api/auth/admin/refresh – Refresh admin session/token
- PUT /api/auth/admin/change-password – Change admin password
- GET /api/auth/me – Retrieve current authenticated user profile

2. Queue Management APIs

These APIs handle queue lifecycle operations including joining, updating, and managing queue entries.

-
- POST /api/queue/join – Join an event queue
 - GET /api/queue/status/{tokenNumber} – Get queue status for a token
 - GET /api/queue – Retrieve full queue list for administrator
 - PUT /api/queue/{id}/start – Mark a queue entry as serving
 - PUT /api/queue/{id}/complete – Mark a queue entry as completed
 - PUT /api/queue/{id}/cancel – Cancel a queue entry
 - PUT /api/queue/{id}/revoke – Revert a queue entry back to waiting

3. Event Management APIs

These APIs allow administrators to manage events and control event lifecycle.

- POST /api/events – Create a new event
- GET /api/events – Retrieve all active or public events
- POST /api/events/{id}/complete – Complete an event and archive it
- DELETE /api/events/{id} – Delete an event and archive it

4. Event History APIs

These APIs provide access to archived event records.

- GET /api/event-history – Retrieve all archived event history
- GET /api/event-history/{id} – Retrieve a specific archived event
- DELETE /api/event-history/{id} – Permanently delete archived record

5. Machine Learning Gateway APIs

These APIs connect the backend with the machine learning microservice.

- POST /api/ml/predict – Unified prediction endpoint
- POST /api/ml/train – Train ML models using queue data
- GET /api/ml/status – Check ML service or model status

6. Prediction Dashboard API

- GET /api/predictions – Retrieve aggregated prediction dashboard data

7. OTP APIs

These APIs manage OTP-based authentication and verification.

- POST /api/otp/send-otp – Send login OTP to user email
- POST /api/otp/verify-otp – Verify OTP and issue access token

8. Utility and Test APIs

These APIs are used for testing and monitoring backend system health.

- GET /api/test-protected – Test protected route access
- GET /api/health – Backend health check
- GET /api/test-email – Trigger test email functionality

Total Backend APIs: The Smart-Q backend exposes a total of **24 HTTP endpoints** covering all major system functionalities.

Internal Machine Learning Microservice APIs

In addition to the main backend APIs, the Smart-Q system includes a separate Python Flask-based machine learning microservice. These APIs are not exposed under the /api namespace and are used internally for prediction and model training.

- POST /queue/joined – Notify ML service when a user joins a queue
- POST /predict/waiting-time – Predict waiting time
- POST /predict/queue-length – Predict queue length
- POST /predict/no-show – Predict no-show probability
- POST /predict/peak-hours – Predict peak-hour density
- POST /suggest/best-time – Suggest optimal visiting time

-
- POST /train – Train machine learning models
 - GET /health – Check ML service health status

These APIs enable advanced predictive capabilities such as waiting time estimation, queue behavior analysis, and intelligent decision support for both users and administrators.

server.js

Code

```
const { startServer } = require("./app");

startServer().catch((error) => {
  console.error("Startup failed:", error.message);
  process.exit(1);
});
```

Explanation

This file acts as the main entry point of the backend application. It imports the server initialization function from the main application module and executes it. If the server fails to start due to configuration issues or database errors, the application terminates immediately. This fail-fast mechanism ensures that the system does not continue running in an unstable state.

app.js

Code

```
app.set("trust-proxy", 1);

app.use(helmet());
```

```
app.use(cors({
  origin(origin, cb) { ... },
  credentials: true
}));

app.use(express.json({ limit: "10kb" }));

app.use("/api", apiLimiter);

app.use("/api/auth", authRoutes);
app.use("/api/queue", queueRoutes);
app.use("/api/events", eventRoutes);
app.use("/api/event-history", eventHistoryRoutes);

const io = new Server(server, { cors: { ... } });

app.set("io", io);
```

Explanation

This file initializes the Express application and configures the main middleware components required by the backend system. Security middleware such as Helmet protects the application against common web vulnerabilities, while CORS enables controlled cross-origin communication between the frontend and backend.

The application also enables JSON request parsing and applies rate limiting to API routes in order to prevent excessive requests. The main API routes responsible for authentication, queue management, event handling, and event history are registered in this module.

Additionally, a Socket.IO server instance is initialized to provide real-time communication between the backend and the connected frontend clients.

config/db.js

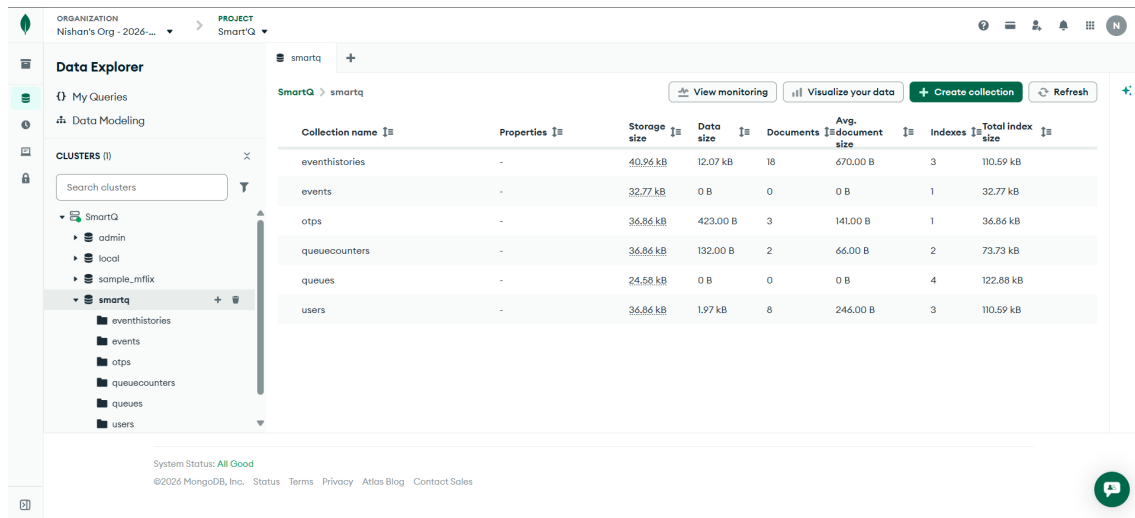


Figure 4.8: Mongo DB Atlas

Code

```
const conn = await mongoose.connect(  
  process.env.MONGO_URI  
);
```

Explanation

This module is responsible for establishing a connection between the application and the MongoDB database using Mongoose. The database connection string is loaded from environment variables, allowing flexible configuration across development, testing, and production environments.

Separating database configuration into a dedicated module improves maintainability and simplifies debugging when database connection issues occur.

routes/authRoutes.js

Code

```
router.post("/register", register);

router.post("/login", loginLimiter, login);

router.post("/admin/login",
loginLimiter,
adminLogin
);

router.put("/admin/change-password",
authMiddleware,
roleMiddleware("admin"),
changePassword
);
```

Explanation

This file defines the API routes responsible for handling user authentication within the Smart-Q system. It includes routes for user registration, user login, and administrator login.

Rate limiting is applied to login endpoints to prevent brute-force attacks. Additionally, sensitive operations such as password changes are protected using authentication and role-based authorization middleware to ensure that only administrators can perform these actions.

middleware/auth.js

Code

```
const decoded = jwt.verify(
token,
```

```
process.env.JWT_ACCESS_SECRET
);

const user =
await User.findById(decoded.id);
```

Explanation

This middleware validates the authentication token provided in client requests. The JSON Web Token (JWT) is verified using a secret key stored in the environment configuration.

After verification, the user identifier embedded in the token is extracted and used to retrieve the corresponding user record from the database. This ensures that only authenticated users can access protected routes.

middleware/rateLimiters.js

Code

```
const otpSendLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 3
});

const queueJoinLimiter = rateLimit({
  windowMs: 10 * 60 * 1000,
  max: 10
});
```

Explanation

This module defines rate limiting rules for specific API endpoints to protect the system from abuse. For example, OTP generation requests are limited to prevent

automated spam attempts, while queue joining requests are restricted to avoid excessive load caused by malicious scripts.

These restrictions help maintain system stability and ensure fair usage of the application.

routes/queueRoutes.js

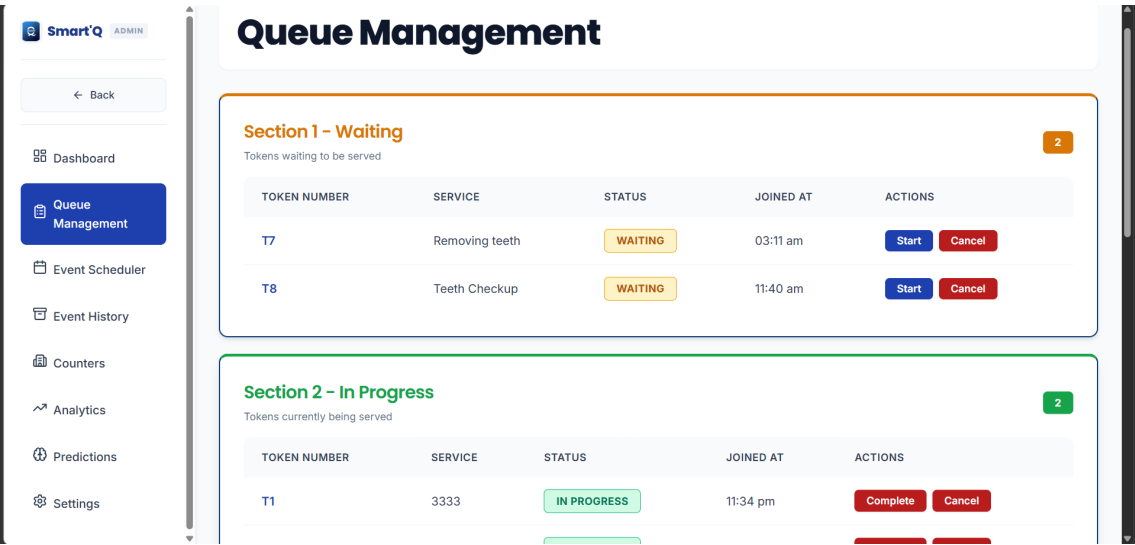


Figure 4.9: Queue Management Interface for Administrators

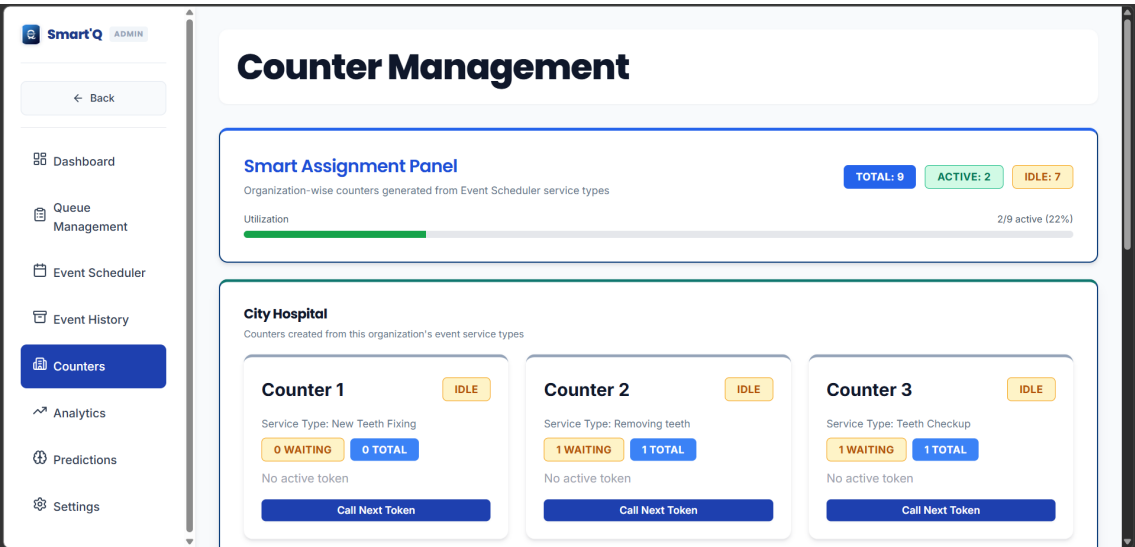


Figure 4.10: Service Counter Management Interface

Code

```
router.post("/join",
queueJoinLimiter,
async (req, res) => {

const tokenNumber =
await getNextTokenNumber(tokenScope);

const newQueue = new Queue({
tokenNumber,
service,
eventId,
organizationType: tokenScope
});

await newQueue.save();

req.app.get("io") &&
await broadcastQueueUpdate(
req.app.get("io")
);

res.status(201).json({
tokenNumber,
position,
estimatedWaitTime
});

});
```

Explanation

When a request is received, the system generates a unique token number and creates

a queue entry in the database with details such as service type, event identifier, and organization category. The backend then broadcasts real-time queue updates to all connected clients via Socket.IO, allowing administrators and customers to track token positions and waiting times. The server finally returns the assigned token number, queue position, and estimated waiting time.

routes/eventRoutes.js

The screenshot shows the 'Event Scheduler' interface within the SmartQ Admin dashboard. The left sidebar contains navigation links: Dashboard, Queue Management, Event Scheduler (highlighted), Event History, Counters, Analytics, Predictions, and Settings. The main content area is titled 'Event Scheduler' and features a 'Schedule New Event' form. The form includes fields for Organization Type (a dropdown), Event Title, Organization Name, and Number of Tokens (with a max of 999). It also has date and time pickers for Start Date, End Date, Start Time, and End Time. A Location field and a Service Types (Manual Entry) section with an 'Add Service' button are also present. A 'Save Event' button is at the bottom.

Figure 4.11: Event Scheduler Interface

The screenshot shows the 'Event History' interface within the SmartQ Admin dashboard. The left sidebar is the same as in Figure 4.11, with 'Event History' highlighted. The main content area is titled 'Event History' and includes a 'Refresh' button. It displays four summary cards: 'TOTAL COMPLETED' (1, All time), 'MANUALLY DELETED' (3, Admin removals), 'AUTO-EXPIRED' (11, Past end date), and 'TOTAL USERS SERVED' (2, Completed across all events). Below these is a search bar and a dropdown for 'All Reasons'. The 'Archived Events' section shows a list of events, including 'Walk-In' and 'Hospital', with details like location, status, and a '15 RECORDS' indicator.

Figure 4.12: Archived Event History Management

Code

```
router.post("/",
  authMiddleware,
  roleMiddleware("admin"),
  async (req, res) => { ... });

router.post("/:id/complete",
  authMiddleware,
  roleMiddleware("admin"),
  async (req, res) => { ... });

router.delete("/:id",
  authMiddleware,
  roleMiddleware("admin"),
  async (req, res) => { ... });
```

Explanation

This module manages event-related operations within the Smart-Q system. It provides routes for creating new events, marking events as completed, and deleting events from the system.

These actions are restricted to administrators through role-based authorization middleware. This ensures that only authorized users can modify event records and manage queue operations associated with those events.

—

services/eventCleanupService.js

Code

```
const expiredEvents =
events.filter(
```

```
(event) => isEventExpired(event, now)
);
```

```
await Queue.deleteMany({
  eventId: { $in: expiredEventIds }
});
```

```
await Event.deleteMany({
  _id: { $in: expiredEventIds }
});
```

Explanation

This service performs automated maintenance tasks by periodically scanning the database for expired events. When an event is detected as expired, all related queue entries are removed and the corresponding event record is deleted or archived.

This cleanup process prevents outdated data from affecting active operations and helps maintain the integrity and performance of the database.

4.3 Machine Learning Implementation

4.3.1 Explanation

The machine learning component is implemented as a separate Python Flask microservice that communicates with the Node.js backend through REST APIs. This architecture allows independent scaling, training, and deployment of machine learning models.

`routes/mlRoutes.js`

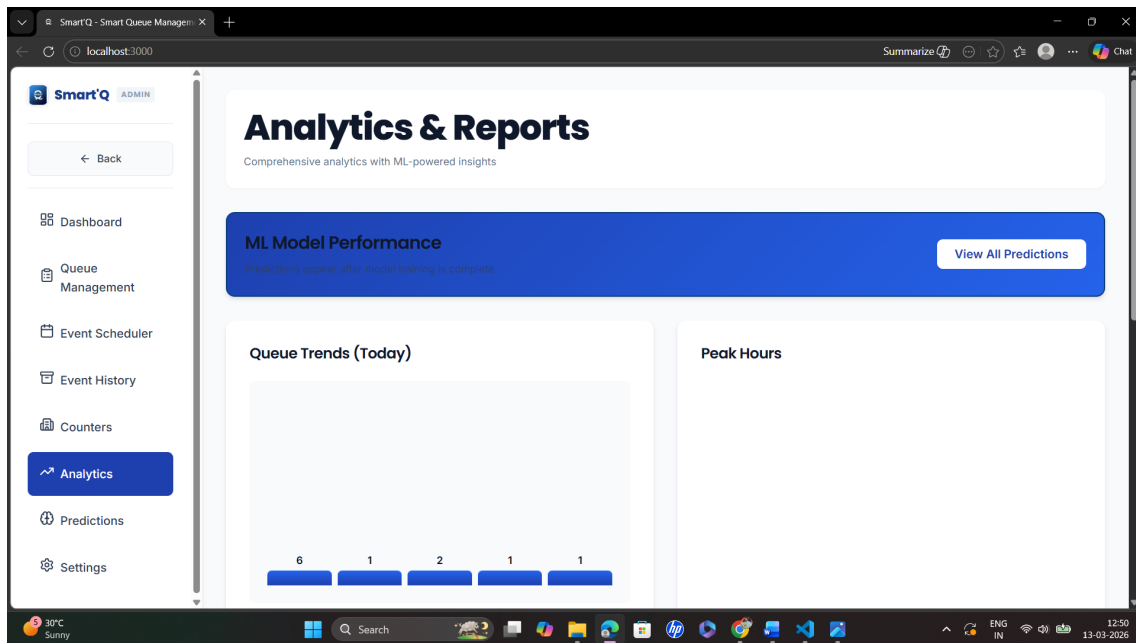


Figure 4.13: Machine Learning Predictions on Customer Dashboard

Code

```
router.post("/predict",
mlPredictLimiter,
async (req, res) => {
```

```
const payloadError =
validatePredictPayload(req.body);

if (payloadError)
return res.status(400)
.json({ message: payloadError });

if (!(await ensureModelTrained())) {
return res.status(409)
.json({ message:
"ML-model-is-not-trained-yet." });
}

const prediction =
await callMLService(endpoint, features);

res.json({ type, prediction, features });

});
```

Explanation

This route validates the input payload and sends prediction requests to the ML microservice.

—

services/mlPredictionService.js

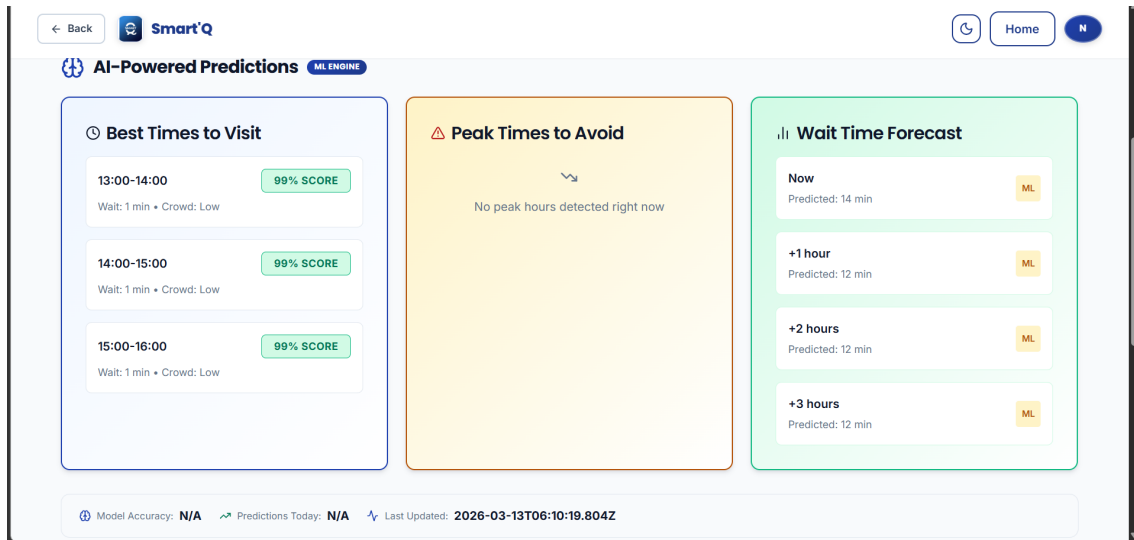


Figure 4.14: Machine Learning Predictions Dashboard

Code

```
if (!health || !health.trained) {

return buildFallbackPredictionBundle({

service ,
positionInQueue ,
totalWaiting ,
reason: "ml-not-trained"

});

}
```

Explanation

This service integrates the backend with the Python ML microservice and provides fallback predictions if the ML model is unavailable.

ml/ml_service.py

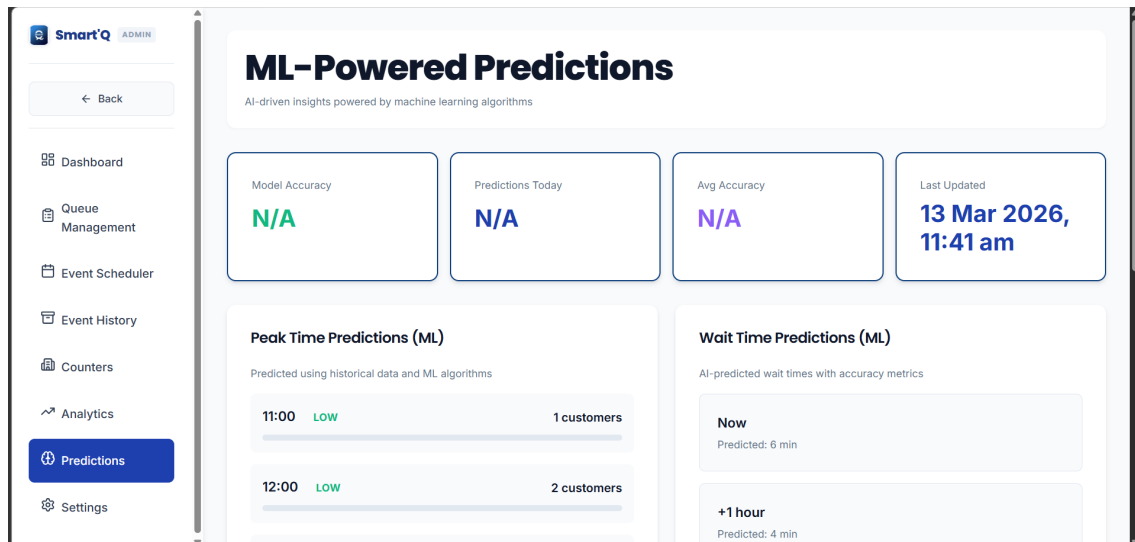


Figure 4.15: Machine Learning Predictions on Customer Dashboard

Code

```
@app.route('/queue/joined', methods=['POST'])
def user_joined():

    record = request.json

    result =
    ml_models.on_user_joined(record)

    return jsonify(result)

@app.route('/predict/waiting-time',
methods=['POST'])

def predict_waiting_time():
```

```
prediction =
ml_models.predict_waiting_time(
request.json
)

return jsonify({
'waitingTime': prediction,
'unit': 'minutes'
})
```

Explanation

This Flask microservice records queue events and generates waiting time predictions using trained machine learning models.

ml/models.py

Code

```
class QueueMLModels:

def __init__(self):

self.waiting_time_model =
RandomForestRegressor(...)

self.no_show_model =
RandomForestClassifier(...)

self._load_all_models()
```

```
def on_user_joined(self ,
queue_record):

    self.buffer.append(queue_record)

if len(self.buffer) >=
RETRAIN_EVERY:

    self._auto_train()
```

Explanation

This module initializes machine learning models and performs automatic retraining when sufficient new queue data is collected.

4.4 Email SMTP Implementation

4.4.1 Explanation

The SMTP module enables transactional email functionality within the system. It supports sending OTP verification emails and queue confirmation messages.

`services/emailService.js`

Code

```
const SMTP_HOST =
process.env.SMTP_HOST ||
"smtp.gmail.com";

const SMTP_PORT =
Number(process.env.SMTP_PORT || 465);

const SMTP_SECURE =
String(process.env.SMTP_SECURE || "true")
.toLowerCase() === "true";

const SMTP_USER =
process.env.SMTP_USER ||
process.env.EMAIL_USER;

const SMTP_PASS =
process.env.SMTP_PASS ||
process.env.EMAIL_PASS;
```

Explanation

This module centralizes email configuration and message sending functionality.

routes/otpRoutes.js

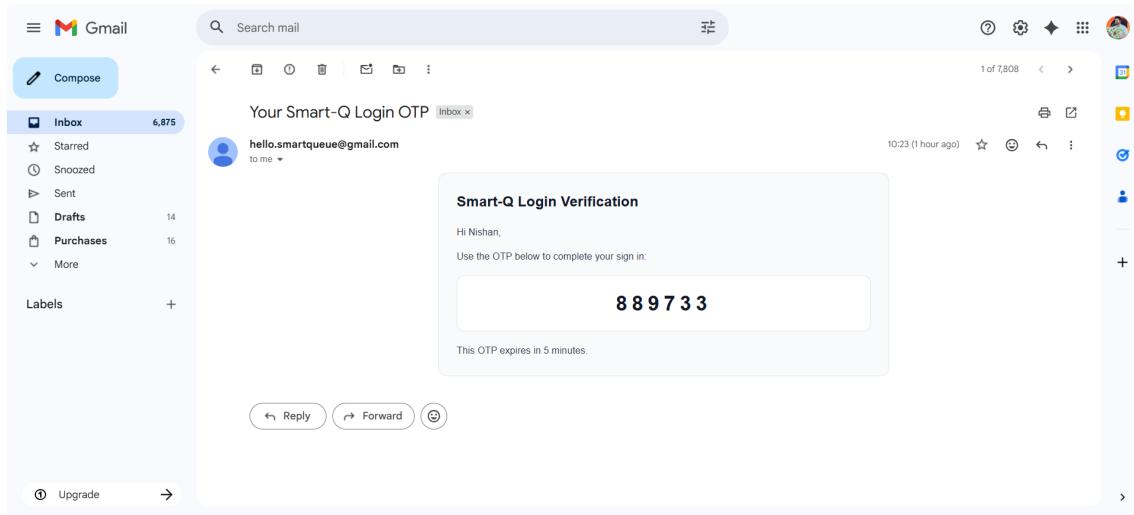


Figure 4.16: OTP Verification Email

Code

```
const mailResult =
await sendLoginOtpEmail({

toEmail: normalizedEmail ,
userName: user.name,
otp

});

if (!mailResult.sent) {

return res.status(500).json({

message: "Failed to send OTP" ,
```

```

reason: mailResult.reason

});

}

```

Explanation

This route handles OTP-based login verification and sends the OTP to the user through the SMTP email service.

routes/queueRoutes.js

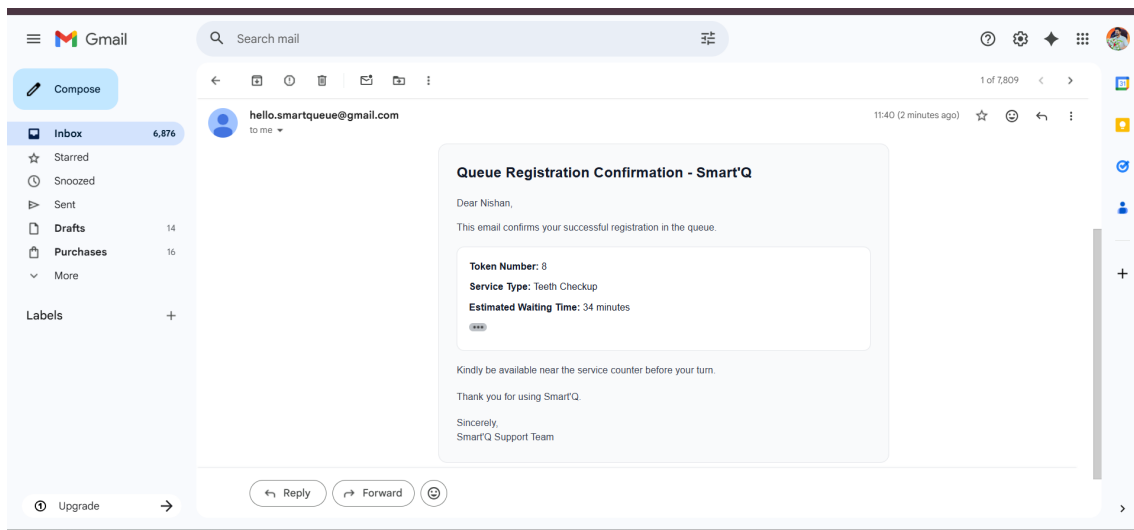


Figure 4.17: Queue Registration Confirmation Email

Code

```

if (isCustomerUser &&
recipientEmail) {

sendQueueRegistrationEmail({

```

```
toEmail: recipientEmail ,
userName: guestName || "User" ,
tokenNumber ,
serviceName: normalizedService ,
estimatedWaitTime

}).catch(( mailError ) => {

console.error(
"Queue - confirmation - email - failed : " ,
mailError.message
);

});

}
```

Explanation

This code sends a queue registration confirmation email containing the token number and estimated waiting time.

CHAPTER 5

Conclusion

The Smart-Q system provides a modern and intelligent solution for managing queues and service events in environments such as hospitals, banks, educational institutions, and government offices. Traditional queue systems often lead to long waiting times, overcrowding, and inefficient service management. The Smart-Q platform addresses these challenges by enabling users to join queues remotely and track their queue position in real time.

The system integrates modern full-stack technologies including React for the frontend, Node.js and Express for backend services, MongoDB for data storage, and Socket.IO for real-time communication. These technologies enable efficient communication between users and administrators while maintaining a responsive and user-friendly interface.

In addition to queue management functionality, the system incorporates a machine learning component that predicts waiting time based on historical queue data. This predictive capability improves user experience by providing accurate waiting time estimates and assists administrators in making better operational decisions. The integration of SMTP email services further enhances the system by enabling automated notifications such as OTP verification and queue confirmation emails.

Overall, the Smart-Q system demonstrates how modern web technologies and machine learning techniques can be combined to develop an intelligent queue management platform. The system improves service efficiency, reduces physical waiting time, and enhances the overall experience for both users and service providers. Future enhancements may include mobile application support, advanced analytics

dashboards, and improved predictive models to further optimize queue management operations.

5.1 System Deployment

The Smart-Q system has been successfully deployed on a cloud hosting platform to provide a live demonstration of the queue and event management application. The deployment enables users and administrators to access the system through a web browser without requiring any local installation.

The deployed application supports all major system functionalities including user authentication, queue registration, real-time queue updates, event management, machine learning based waiting time prediction, and automated email notifications.

The live version of the Smart-Q system can be accessed using the following deployment link:

Website Deployment Link:

<https://smartq-system.vercel.app/>

This online deployment allows users to experience the complete workflow of the system including queue joining, monitoring queue status, and receiving notifications.

5.2 Source Code Repository

The complete source code of the Smart-Q system is maintained using GitHub version control. The repository contains the frontend application, backend server implementation, machine learning microservice, and configuration files required for deployment.

Using GitHub allows collaborative development, version tracking, and easier maintenance of the project codebase.

The project repositories can be accessed through the following links:

Frontend Repository:

<https://github.com/abhaykrishnag/Smart-Q/tree/main/frontend>

Backend Repository:

<https://github.com/abhaykrishnag/Smart-Q/tree/main/backend>

Machine Learning Service Repository (Optional):

<https://github.com/abhaykrishnag/Smart-Q/tree/main/backend/ml>

These repositories contain the complete implementation of the Smart-Q system, including frontend interfaces, backend APIs, machine learning models, and email notification services.

BIBLIOGRAPHY

1. Google Developers, *Frontend Web Development*. Available at: <https://developers.google.com/web>
2. Net Ninja, *React Tutorial for Frontend*, YouTube Video. Available at: <https://www.youtube.com/watch?v=j942wKiXFu8>
3. Normalized Nerd, *Machine Learning (Random Forest Algorithm)*, YouTube Video. Available at: <https://www.youtube.com/watch?v=v6VJ2RO66Ag&t=1s>
4. TurboSMTP, *How to Configure an SMTP Server*. Available at: <https://serversmtp.com/smtp-configuration/>
5. Caleb Curry, *REST API Crash Course: Introduction and Full Python API Tutorial*, YouTube Video. Available at: <https://youtu.be/qbLc5a9jdXo?si=BUELpKQAEcJOfNw1>
6. Anthropic, *Building with the Claude API*. Available at: <https://anthropic.skilljar.com/claude-with-the-anthropic-api>
7. MongoDB, *MongoDB Development Documentation*. Available at: <https://www.mongodb.com/docs/development/>
8. OpenAI, *Build Your Server (Apps SDK MCP Server)*. Available at: <https://developers.openai.com/apps-sdk/build/mcp-server>

APPENDICES

Appendix A: Website Screenshots

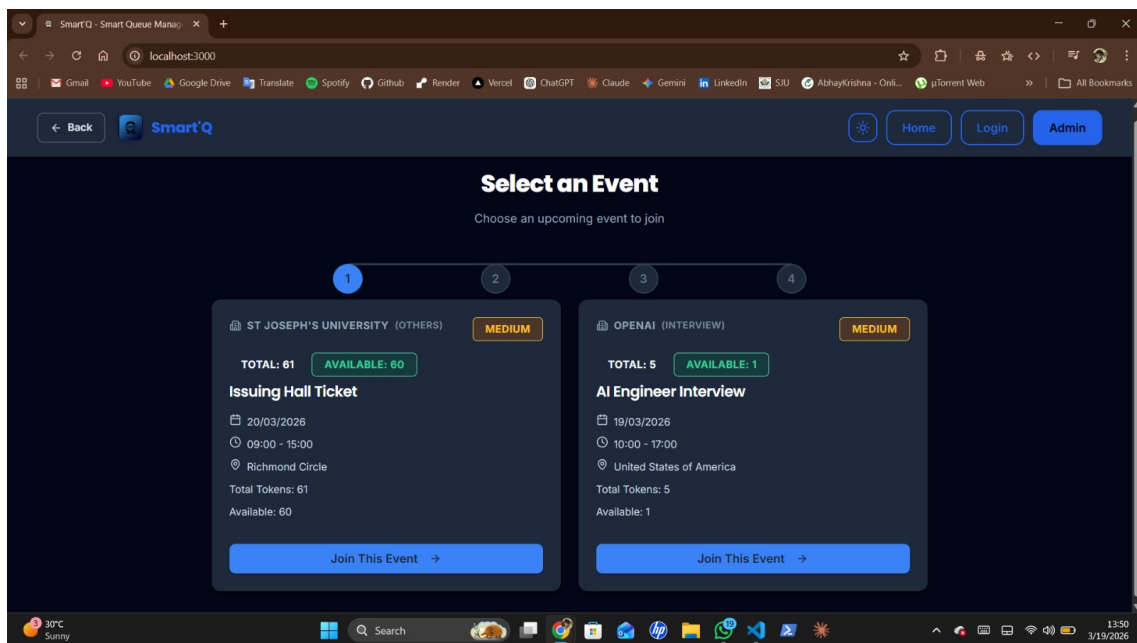


Figure 5.1: Smart-Q Guest Mode Dashboard Page

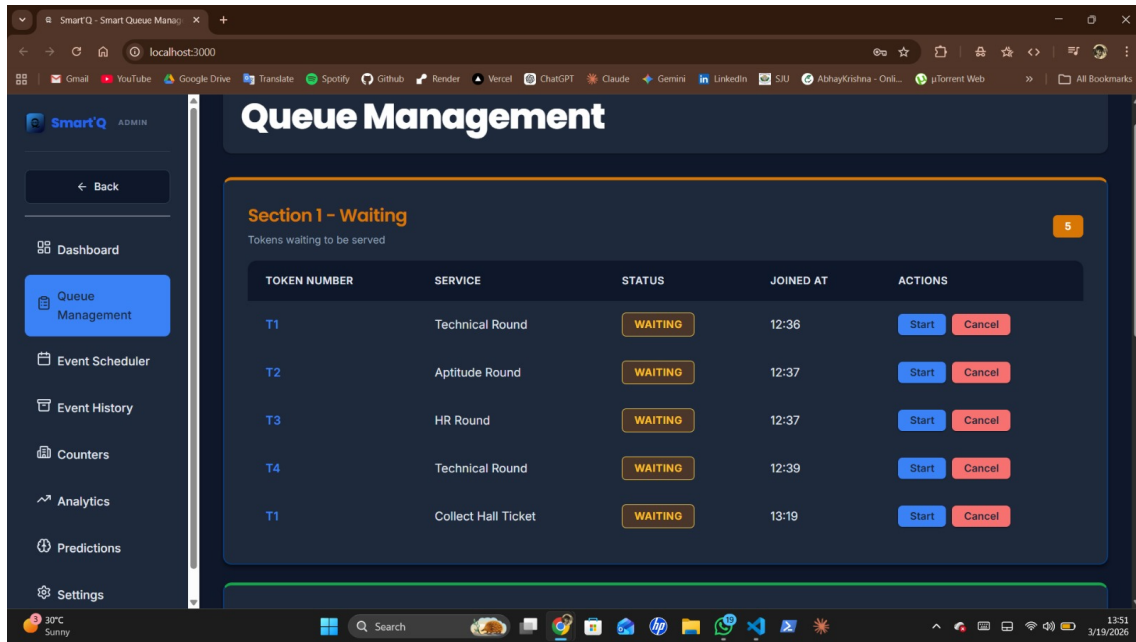


Figure 5.2: Smart-Q Waiting Queue Management Page

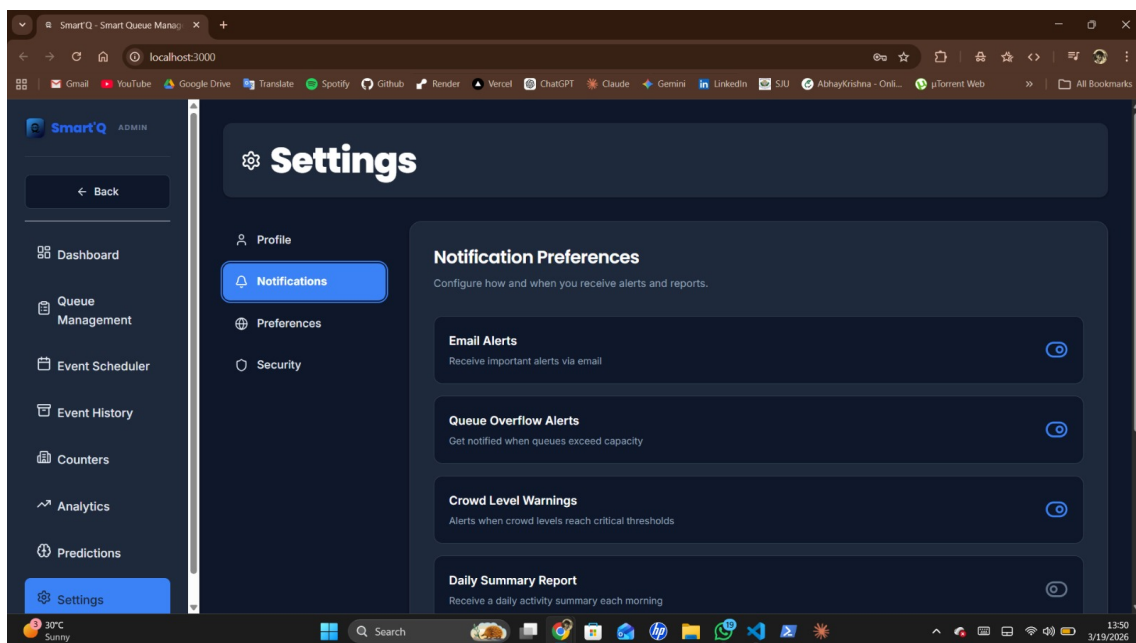


Figure 5.3: Admin Notification Interface

Appendix B: Code Screenshots

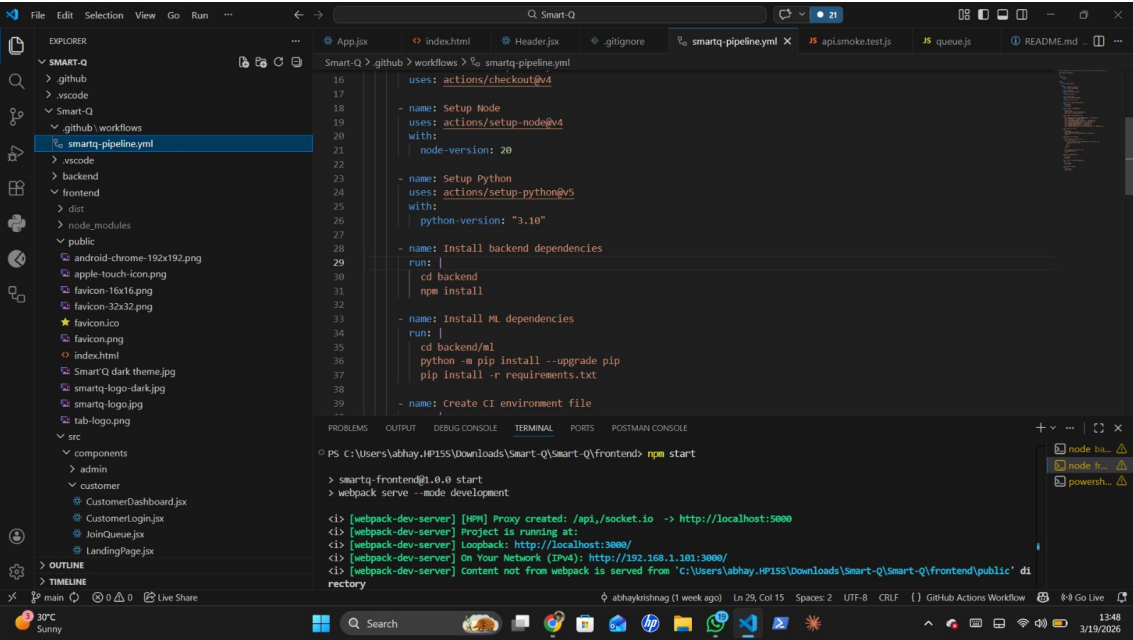


Figure 5.4: CI/CD Pipeline Configuration for Smart-Q Deployment

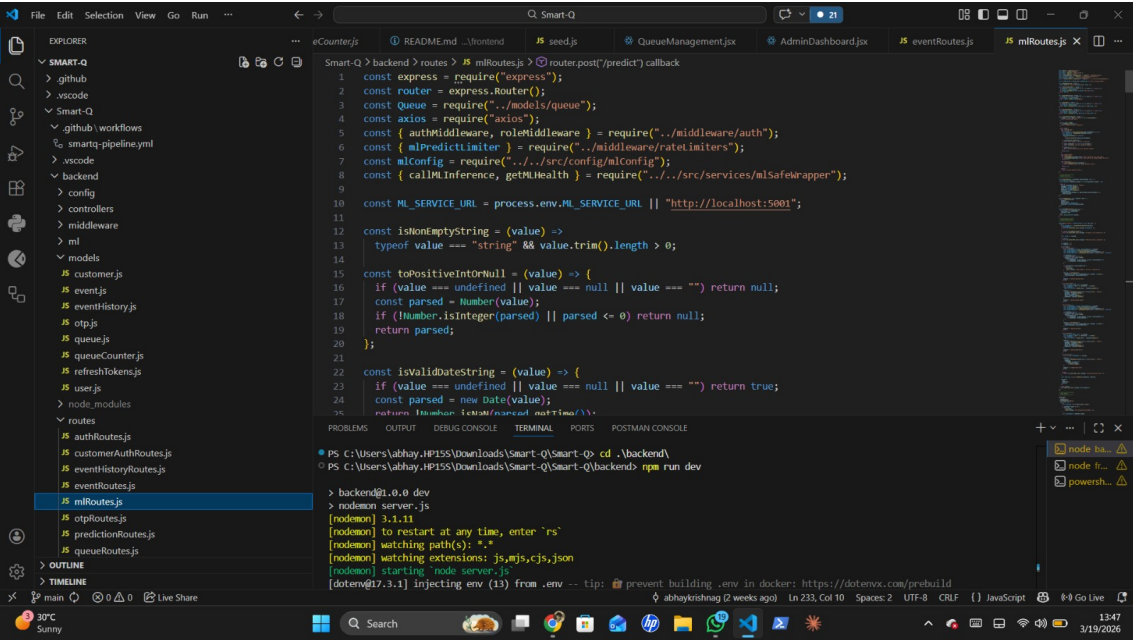


Figure 5.5: Backend Routes for Machine Learning Prediction